

La Cave du Coin — Référence des scripts du projet

Rôle de chaque script présent dans scripts/, ses dépendances vis-à-vis des autres fichiers/scripts, et l'ordre dans lequel ils s'exécutent selon le scénario (déploiement initial, vie courante, rollback, documentation).

1. Vue d'ensemble par catégorie

Les scripts se répartissent en cinq familles. Toutes s'exécutent sur le VPS depuis le dossier du projet (/opt/La_Cave_du_Coin), sauf register_backup_task.ps1 qui est un script Windows destiné à un PC hébergeant lui-même l'application.

- **Initialisation / configuration** — seed_data.py, add_pgadmin_env_vars.sh, add_pgadmin_domain.sh.
- **Sauvegarde** — backup.py, register_backup_task.ps1.
- **Restauration / rollback** — restore_prod_from_backup.sh, restore_dump_into_staging.sh, restore_latest_reset_backup_into_staging.sh.
- **Environnement staging** — refresh_staging_from_prod.sh, verify_staging.sh.
- **Réinitialisation prod** — reset_prod_database.sh (script pivot : produit un fichier consommé par deux scripts de la famille "staging").
- **Génération de documentation PDF** — les cinq scripts generate_*_pdf.py, indépendants les uns des autres.

2. Initialisation et configuration

Script	Rôle et dépendances
scripts/seed_data.py	Crée les 4 comptes par défaut (superadmin/admin/manager/caissier) et, sauf appel avec --users-only, 8 produits de démonstration. Prérequis : le schéma de base doit déjà exister (fait automatiquement par alembic upgrade head au démarrage du conteneur app, cf. Dockerfile). Appelé par : reset_prod_database.sh (en mode --users-only) ; peut aussi être lancé seul lors d'un premier déploiement.
scripts/add_pgadmin_env_vars.sh	Ajoute à .env.staging les identifiants pgAdmin par défaut (email + mot de passe), sans avoir à taper le caractère @ à la console Hetzner. Idempotent (sans effet si déjà fait). Prérequis : être lancé depuis le dossier du projet, .env.staging doit déjà exister.
scripts/add_pgadmin_domain.sh	Ajoute à .env (celui de prod, lu par Caddy) la ligne PGADMIN_DOMAIN=..., sans taper de _. Idempotent. Effet dépendant : un docker compose up -d --build est nécessaire ensuite pour que Caddy relise le .env mis à jour.

3. Sauvegarde

Script	Rôle et dépendances
scripts/backup.py	Sauvegarde la base (SQLite en local, ou PostgreSQL en prod via docker compose exec db pg_dump) vers un fichier SQL brut horodaté dans backups/, copie éventuellement hors-site, puis purge les sauvegardes plus vieilles que BACKUP_RETENTION_DAYS. Format produit : SQL brut (pg_dump sans -Fc) — consommé par restore_prod_from_backup.sh uniquement (pas compatible avec les scripts qui attendent un format -Fc, voir encadré section 5).

<code>scripts/register_backup_tasks.ps1</code>	Script Windows (PowerShell, à exécuter une fois en administrateur) qui enregistre une tâche planifiée exécutant <code>backup.py</code> toutes les 4h. Dépend de : un environnement virtuel Python déjà créé (<code>.venv</code>). Pertinent seulement si l'application tourne sur un PC local (option non retenue pour ce projet, hébergé sur VPS) — sur le VPS, l'équivalent serait une tâche cron appelant <code>backup.py</code> .
--	---

4. Réinitialisation de la production

Script	Rôle et dépendances
<code>scripts/reset_prod_database.sh</code>	Script pivot à usage ponctuel : sauvegarde la prod actuelle (<code>pg_dump -Fc → backup_before_reset_<horodatage>.dump</code>), reconstruit l'image app, vide toutes les tables métier (<code>TRUNCATE ... RESTART IDENTITY CASCADE</code>), puis appelle <code>seed_data.py --users-only</code> pour ne recréer que les 4 comptes par défaut. Format produit : <code>dump -Fc</code> (binaire compressé, nécessite <code>pg_restore</code> , pas <code>psql <</code>). Consommé par : <code>refresh_staging_from_prod.sh</code> (comme solution de repli si la prod est encore vide) et <code>restore_latest_reset_backup_into_staging.sh</code> (pour renvoyer ces données de démo vers staging).

Point d'attention (rencontré en pratique le 2026-09-28) : le fichier produit par ce script (`-Fc`) n'est **pas** compatible avec `restore_prod_from_backup.sh`, qui attend un dump SQL brut (celui de `backup.py`). Pour restaurer un fichier `backup_before_reset_*.dump` directement en prod (cas rare, hors staging), il faut utiliser manuellement `pg_restore` avec les options `--clean --if-exists`, comme le fait déjà `restore_dump_into_staging.sh` mais visé sur la base prod plutôt que staging.

5. Restauration / rollback

Script	Rôle et dépendances
<code>scripts/restore_prod_from_backup.sh <fichier.sql></code>	Restaure en production un dump SQL brut (celui produit par <code>backup.py</code>). Prend automatiquement une sauvegarde de sécurité de l'état actuel avant toute modification (<code>backups/avant_rollback_<horodatage>.sql</code>), arrête l'app, recrée la base à vide, charge le dump, redémarre l'app. Dépend de : un fichier produit par <code>backup.py</code> — n'accepte pas les <code>.dump</code> de <code>reset_prod_database.sh</code> .
<code>scripts/restore_dump_into_staging.sh <fichier.dump></code>	Restaure dans la base staging (jamais en prod) un dump au format <code>-Fc</code> via <code>pg_restore --clean --if-exists</code> . Dépend de : le stack staging doit être démarré. Appelé par : <code>restore_latest_reset_backup_into_staging.sh</code> .
<code>scripts/restore_latest_reset_backup_into_staging.sh</code>	Détecte automatiquement le fichier <code>backup_before_reset_*.dump</code> le plus récent (évite de taper son nom, risqué à la console Hetzner à cause des underscores) et appelle <code>restore_dump_into_staging.sh</code> avec ce chemin. Dépend directement de : un fichier déjà produit par <code>reset_prod_database.sh</code> .

6. Environnement staging

Script	Rôle et dépendances
--------	---------------------

<code>scripts/refresh_staging_from_prod.sh</code>	Script principal de maintenance de l'environnement de test ISO prod : git pull puis reconstruction du stack staging (docker compose -p staging ... up -d --build), puis copie des données. Si la prod contient déjà de vraies données (produits + ventes > 0), copie normale prod → staging (pg_dump pg_restore) ; sinon, si un fichier backup_before_reset_*.dump existe, restaure celui-ci à la place (pour ne pas laisser staging vide juste après un reset_prod_database.sh). Appelé automatiquement chaque mois par une tâche cron. Suivi par : verify_staging.sh.
<code>scripts/verify_staging.sh</code>	Vérifie que le rafraîchissement a fonctionné : affiche les utilisateurs et produits présents dans staging-db-1. Dépend de : le stack staging démarré et déjà rafraîchi une fois.

7. Génération de documentation PDF

Ces cinq scripts sont indépendants les uns des autres et ne dépendent d'aucun autre script de ce projet — seulement de la présence de reportlab et, pour le premier, du contenu de FORMATION.md. À relancer manuellement après toute mise à jour du contenu qu'ils reflètent.

Script	Rôle et dépendances
<code>scripts/generate_formation_slides_pdf.py</code>	Diaporama PDF (une diapo par page, paysage) condensé à partir de FORMATION.md, pour une présentation en réunion. Dépend de : le contenu de FORMATION.md (à tenir à jour en amont).
<code>scripts/generate_project_overview_pdf.py</code>	Présentation technique complète du projet (stack, architecture, tests, hébergement, sécurité), pensée comme support de préparation à un entretien.
<code>scripts/generate_session_summary_pdf.py</code>	Récapitulatif figé d'une séance de travail précise (2026-09-15) — mise en place de l'accès pgAdmin à la vraie base de prod.
<code>scripts/generate_staging_setup_pdf.py</code>	Explique, commande par commande, la mise en place de l'environnement staging + pgAdmin.
<code>scripts/generate_test_scenario_pdf.py</code>	Scénario de test manuel couvrant crédit client, créances, sessions de caisse.

8. Ordre d'exécution selon le scénario

8.1 — Déploiement initial sur un nouveau serveur

1. Configuration (add_pgadmin_env_vars.sh, add_pgadmin_domain.sh si pgAdmin voulu) → 2. docker compose up -d --build (le schéma se crée automatiquement, alembic upgrade head tourne au démarrage du conteneur) → 3. seed_data.py (comptes + démo, ou --users-only si aucune démo voulue) → 4. (si PC local) register_backup_task.ps1 → 5. refresh_staging_from_prod.sh puis verify_staging.sh si un environnement de test est voulu.

8.2 — Vie courante

backup.py s'exécute seul, automatiquement, toutes les 4h. refresh_staging_from_prod.sh s'exécute seul, automatiquement, une fois par mois. Aucune action manuelle nécessaire tant que tout fonctionne normalement.

8.3 — Passage aux données réelles (fait le 2026-09-28)

1. `reset_prod_database.sh` (vide la prod, produit `backup_before_reset_*.dump`) → 2. (optionnel) `restore_latest_reset_backup_into_staging.sh` si on veut garder les données de démo consultables dans l'environnement de test plutôt que de les perdre au prochain rafraîchissement mensuel.

8.4 — Rollback en cas de problème

Selon l'origine du fichier de sauvegarde à restaurer : `restore_prod_from_backup.sh` pour un fichier produit par `backup.py` (SQL brut) ; `pg_restore --clean --if-exists manuel` pour un fichier -Fc (produit par `reset_prod_database.sh`) qu'on voudrait exceptionnellement renvoyer en prod plutôt qu'en staging.

9. Schéma des dépendances entre scripts

Les flèches indiquent "produit un fichier consommé par" ou "appelle" :

- `reset_prod_database.sh` → produit `backup_before_reset_*.dump` → consommé par `refresh_staging_from_prod.sh` (repli) **et** `restore_latest_reset_backup_into_staging.sh`.
- `restore_latest_reset_backup_into_staging.sh` → appelle → `restore_dump_into_staging.sh`.
- `reset_prod_database.sh` → appelle → `seed_data.py --users-only`.
- `backup.py` → produit un fichier `backups/*.sql` → seul consommateur : `restore_prod_from_backup.sh`.
- `register_backup_task.ps1` → planifie l'exécution récurrente de → `backup.py`.
- `refresh_staging_from_prod.sh` → suivi habituellement de → `verify_staging.sh`.
- Les scripts `generate_*.pdf.py` et `add_pgadmin_*.sh` n'ont aucune dépendance vers un autre script de ce projet.